Document number: P0702R1
Date: 2017-07-14
Reply-To: Mike Spertus, Symantec (mike_spertus@symantec.com)
Wording by Jason Merrill, Red Hat (jason@redhat.com)
Audience: {Evolution, Core} Working Group

Language support for Constructor Template Argument Deduction

Introduction

This paper details some language considerations that emerged in the course of integrating Class Template Argument Deduction into the standard library as adopted in p0433r2 on which we hope to receive clarification from the committee.

List vs direct initialization

The current standard wording implies the following:

```
tuple t{tuple{1, 2}};   // Deduces tuple<int, int>
vector v{vector{1, 2}}; // Deduces vector<vector<int>>
```

We find it seems inconsistent and difficult to teach that `vector` prefers deduction from the list constructor while such similar code for `tuple` prefers the copy constructor. In Kona, EWG voted (wg21-only link) to prefer copying, but it is not clear whether the intent was that this apply to cases like `vector` as well.

We would like EWG to clarify what was intended in this case and, if necessary, apply any change as a DR. In light of the example above as well as §11.6.4p3.8 [dcl.init.list], we recommend that the copy deduction candidate be preferred to the list constructor when initializing from a list consisting of a single element if it would deduce a type that is reference-compatible with the argument.

EWG voted in Toronto that cases like `vector{vector{1, 2, 3}}` should be vector<int> even for classes that have an initializer_list constructor: 9 | 9 | 11 | 4 | 1

Proposed wording

Modify 16.3.1.8:

> When resolving a placeholder for a deduced class type (10.1.7.5) where the *template-name* names a primary class template C, A set of functions and function templates is formed comprising:
>
> - For each constructor of ~~the primary class template designated by the template-name~~ C , if ~~the template~~ C is defined, a function template with the following properties:
> - The template parameters are the template parameters of ~~the class template~~ C followed by the template parameters (including default template arguments) of the constructor, if any.
> - The types of the function parameters are those of the constructor.
> - The return type is the class template specialization designated by ~~the template-name~~ C and template arguments corresponding to the template parameters ~~obtained from the class template~~ of C .

- If ~~the primary class template~~ C is not defined or does not declare any constructors, an additional function template derived as above from a hypothetical constructor C().
- ...

Initialization and overload resolution are performed as described in 11.6 and 16.3.1.3, 16.3.1.4, or 16.3.1.7 (as appropriate for the type of initialization performed) for an object of a hypothetical class type, where the selected functions and function templates are considered to be the constructors of that class type for the purpose of forming an overload set, and the initializer is provided by the context in which class template argument deduction was performed. As an exception, the first phase in 16.3.1.7 (considering initializer-list constructors) is omitted if the initializer list consists of a single expression of type cv U, where U is a specialization of C or a class derived from a specialization of C. Each such notional constructor is considered to be explicit if the function or function template was generated from a constructor or deduction-guide that was declared explicit. All such notional constructors are considered to be public members of the hypothetical class type.